

Actividad Integradora TypeScript

Food Store

Programación III

Coordinador: Enferrel, Ariel
Profesor/es: Rigoni, Cinthia
Tutor/a:
Alumno/a: Lauk, Karen

Objetivo

Evolucionar la aplicación dinámica "Food Store" hacia un sistema con autenticación y Roles. El alumno deberá implementar un flujo de seguridad que proteja el contenido según el tipo de usuario, sustituyendo la carga abierta de datos por un acceso restringido mediante TypeScript y localStorage.

CONSIGNA

- Tipado Fuerte (TS) → Uso de Interfaces para asegurar que el usuario y sus roles (admin o client) sigan una estructura rígida y sin errores.
- Persistencia (Local) → Uso de localStorage para simular una base de datos de usuarios y mantener la sesión activa del navegador
- Autenticación → Proceso de verificación de credenciales (Email y Password) comparándolos contra el almacenamiento local.
- Autorización (Roles) → Lógica que decide si un usuario puede acceder a una ruta específica (ej. /admin/) basándose en su rol.

Caso Práctico

Actualmente, el repositorio base https://github.com/chiro45/proteger_rutas tiene una estructura de carpetas que debes respetar:

- /pages: Contenedores de las vistas (auth, admin, client).
- /utils: Lógica de navegación y verificación de sesión.
- /types: Definición de contratos de datos (IUser, Rol).

PASO 1: Registro de Usuarios (src/pages/auth/registro/)

Debes transformar el formulario estático en un sistema de captación de datos.

- Modificar el HTML para incluir: Email, contraseña y quitar el selector de rol.

Modificamos el registro en un sistema funcional, simplificando la entrada de datos eliminando la selección manual de roles y centrando la captura en credenciales. →

- En el archivo .ts, capturar los datos y guardarlos en un Array de Objetos dentro del localStorage bajo la clave "users".

Se gestiona la captura y la persistencia de datos utilizando la Web Storage API.

```
<body>
  <div class="modal-overlay">
    <!-- Logo -->
    

    <form id="formRegistro" class="register-box">

      <h1>Registro de usuario</h1>

      <label for="email">Email:</label>
      <input
        type="email"
        id="email"
        name="email"
        autocomplete="email"
        required
      />

      <label for="password">Password:</label>
      <input
        type="password"
        id="password"
        name="password"
        autocomplete="new-password"
        required
      />
      <br>
      <button type="submit">Registrarse</button>

    </form>

    <p><a href="/index.html">Volver al inicio</a></p>
  </div>
```

El flujo de registro:

1. Captura → Recuperan los valores de los inputs mediante un objeto de datos.
2. Validación de Existencia → Antes de guardar, el sistema consulta la clave users en el localStorage.
3. Persistencia: * Si no existen usuarios, se inicializa el Array.
 - a. Si existen, se añade el nuevo objeto al Array actual.
 - b. Se convierte el Array a formato JSON para su almacenamiento.

Propiedad	Tipo	Descripción
email	string	Identificador único del usuario.
password	string	Credencial de acceso (almacenada en texto plano para esta versión).

```
import type { IUser } from "../../types/IUser";

//Estructura general - Espera que la página HTML cargue completamente
document.addEventListener("DOMContentLoaded", () => {
  const form = document.getElementById("formRegistro") as HTMLFormElement | null;

  if (!form) {
    console.error("✗ No se encontró el formulario");
    return;
  }

  form.addEventListener("submit", (e) => {
    e.preventDefault();

    // inputs
    const emailInput = document.getElementById("email") as HTMLInputElement | null;
    const passwordInput = document.getElementById("password") as HTMLInputElement | null;

    if (!emailInput || !passwordInput) {
      console.error("✗ Inputs no encontrados");
      return;
    }

    const email = emailInput.value.trim();
    const password = passwordInput.value.trim();

    // traer usuarios existentes
    const users: IUser[] = JSON.parse(localStorage.getItem("users") || "[]");

    // evitar duplicados
    const exists = users.some(u => u.email === email);

    if (exists) {
      console.log("⚠ Este usuario ya existe");
      return;
    }

    // crear usuario
    const newUser: IUser = {
      email,
      password,
      rol: "client"
    };

    // guardar
    users.push(newUser);
    localStorage.setItem("users", JSON.stringify(users));

    console.log("✓ Usuario guardado:", users);

    // limpiar form
    form.reset();
  });
});
```

PASO 2: Login y Gestión de Sesión (src/pages/auth/login/)

El login ya no debe ser una simulación; debe validar datos reales.

- Al intentar ingresar, buscar en el array de "users" si existe una coincidencia de email y contraseña.

Se ha implementado una validación de credenciales en tiempo real que consulta la "base de datos" local. El acceso ya no es simulado; requiere una coincidencia exacta con los registros creados en el Paso 1.

```
4 <head class="header">
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Login</title>
8
9   <link rel="stylesheet" href="login.css">
10  <script type="module" src="/src/pages/auth/login/login.ts"></script>
11  <script type="module" src="/src/main.ts"></script>
12 </head>
13
14 <body>
15   
16
17   <form id="formLogin" class="login-box">
18     <h1>Iniciar sesión</h1>
19     <div>
20       <label for="email">Email</label>
21       <input type="email" id="email" name="email" autocomplete="off" placeholder="example@example.com">
22     </div>
23
24     <div>
25       <label for="password">Contraseña</label>
26       <input type="password" id="password" name="password" autocomplete="off" placeholder="*****">
27     </div>
28
29     <button type="submit">Ingresar</button>
30     <p id="error" style="color: red;"></p>
31   </form>
32
33   <p><a href="/index.html">Volver al inicio</a></p>
34 </body>
35 </html>
```

El formulario utiliza selectores específicos que permiten la manipulación mediante el DOM:

Identificadores Clave: formLogin, email, password.

Se incluye un elemento <p id="error"> para mostrar mensajes de validación fallida sin interrumpir el flujo con alertas intrusivas.

- Si es correcto, guardar el objeto del usuario en la clave "userData" para iniciar la sesión.

El sistema de login actúa como el guardián de la aplicación. Su función es doble: autenticar (confirmar que el usuario es quien dice ser) y autorizar (dirigir al usuario al área que le corresponde según su rol).

El flujo de ejecución:

1. Interceptación del Evento: Captura el submit del formulario y previene la recarga de página.
2. Consulta a la "Base de Datos": Recupera el array de users del localStorage.

3. Validación de Credenciales: * Utiliza el método `.find()` para buscar una coincidencia exacta entre el email y password ingresados frente a los registros existentes.
4. Gestión de Errores: Si no hay coincidencia, manipula el DOM para mostrar un mensaje de error en el elemento `#error`.
5. Persistencia de Sesión Real: Al encontrar al usuario, guarda el objeto completo (incluyendo su rol asignado en el registro) en la clave `userData`.

```
1  import type { IUser } from "../../types/IUser";
2
3  document.addEventListener("DOMContentLoaded", () => {
4
5      const form = document.getElementById("formLogin") as HTMLFormElement | null;
6      const error = document.getElementById("error") as HTMLParagraphElement | null;
7
8      if (!form) return;
9
10     form.addEventListener("submit", (e) => {
11         e.preventDefault();
12
13         const emailInput = document.getElementById("email") as HTMLInputElement | null;
14         const passwordInput = document.getElementById("password") as HTMLInputElement | null;
15
16         if (!emailInput || !passwordInput) return;
17
18         const email = emailInput.value.trim();
19         const password = passwordInput.value.trim();
20
21         // traer usuarios registrados
22         const users: IUser[] = JSON.parse(localStorage.getItem("users") || "[]");
23
24         // buscar usuario
25         const userFound = users.find(
26             user => user.email === email && user.password === password
27         );
28
29         // ✖ si no existe
30         if (!userFound) {
31             if (error) error.textContent = "✖ Usuario o contraseña incorrectos";
32             return;
33         }
34
35         // ✔ guardar sesión REAL (sin hardcodear rol)
36         localStorage.setItem("userData", JSON.stringify(userFound));
37
38         console.log("✔ Login correcto:", userFound);
39
40         // 🚀 redirección por rol real
41         if (userFound.rol === "admin") {
42             window.location.href = "/src/pages/admin/home/home.html";
43         } else {
44             window.location.href = "/src/pages/client/home/home.html";
45         }
46     });
47 }
```

PASO 3: Protección de Rutas (El "Guard")

Implementar la lógica para que las páginas no sean accesibles mediante URL si no hay sesión.

- Centralización: En src/main.ts, crear una función que intercepte la carga de la página.
- Validación: Si un usuario con rol client intenta entrar a una carpeta /admin/, debe ser redirigido automáticamente al login o a su zona permitida.

Este script actúa como un guardián de seguridad global. Su objetivo es interceptar la navegación y validar que el usuario tenga una sesión activa y el rol adecuado antes de permitir que el contenido de la página sea visible. Al estar centralizado en main.ts, se garantiza una política de seguridad uniforme en toda la aplicación.

Lógica de Restricción (Guard Rules)

El flujo de control se divide en tres niveles de seguridad jerárquicos:

1. Control de Autenticación

Si el sistema no detecta la clave userData en el localStorage, se activa un bloqueo total para cualquier ruta privada.

2. Control de Autorización: Cliente

Protege la integridad del área administrativa contra accesos no autorizados.

3. Control de Autorización: Admin

Mantiene la experiencia del administrador enfocada en su panel de gestión.

```

1  import type { IUser } from "../types/IUser";
2
3  document.addEventListener("DOMContentLoaded", () => {
4      const userData: IUser | null = JSON.parse(
5          localStorage.getItem("userData") || "null"
6      );
7
8      const path = window.location.pathname;
9
10     // --- 1. DEFINIR PÁGINAS PÚBLICAS (LIBRE ACCESO) ---
11     // Agregamos "/" y "index.html" para que la fachada sea visible sin login
12     const isPublicPage = path === "/" ||
13         path.endsWith("index.html") ||
14         path.includes("login.html") ||
15         path.includes("registro.html");
16
17     // --- 2. PROTECCIÓN DE SESIÓN ---
18     if (!userData) {
19         // Si no hay usuario y NO está en una página pública, al Login
20         if (!isPublicPage) {
21             window.location.href = "/src/pages/auth/login/login.html";
22         }
23         return; // Permitir el acceso a la fachada, login o registro
24     }
25
26     // --- 3. REDIRECCIÓN SI YA ESTÁ LOGUEADO ---
27     // Si ya tiene sesión e intenta entrar a login/registro, lo mandamos a su home
28     if (path.includes("login.html") || path.includes("registro.html")) {
29         const home = userData.rol === "admin"
30             ? "/src/pages/admin/home/home.html"
31             : "/src/pages/client/home/home.html";
32         window.location.href = home;
33         return;
34     }
35
36     // --- 4. CONTROL DE ROLES ---
37     if (userData.rol === "client" && path.includes("/admin/")) {
38         window.location.href = "/src/pages/client/home/home.html";
39         return;
40     }
41
42     if (userData.rol === "admin" && path.includes("/client/")) {
43         window.location.href = "/src/pages/admin/home/home.html";
44         return;
45     }
46 });

```

Parte visual

Index.html sin logearse



- [Productos](#)
- [Categorías](#)
- [Ingresar](#)
- [Crear cuenta](#)

Las mejores hamburguesas a un clic.

Explora nuestro menú y regístrate para realizar tu pedido.

Categorías

Hamburguesas
Pizzas
Guarniciones
Ensaladas
Empadanas
Tartas
Bebidas



Nuestras Smash Burgers

Inicia sesión para ver precios y comprar.

[Ver catálogo completo](#)

Login

Iniciar sesión

Email

example@example.com

Contraseña

Ingresar

¿No tienes una cuenta? [Regístrate aquí](#)

Registro



Registro de usuario

Email:

Password:

Registrarse

¿Ya tienes cuenta? [Inicia sesión](#)

FoodStore - ya logueado



[Inicio](#)

[Productos](#)

[Carrito](#)



¡Hola, Usuario!

[Cerrar sesión](#)



Nuestros productos

[Ver todos >](#)



Hamburguesa Triple

Hamburguesa triple smash con mucho cheddar.

\$25.000

Agregar al carrito



Pizza Muzarella

Salsa de tomate casera y muzarella abundante.

\$18.000

Agregar al carrito



Papas Fritas

Cortada en bastones y sin sal.

\$7.500

Agregar al carrito



Bebidas

Opción a elegir: Coca-cola, Sprite, Pepsi y Fanta.

\$1.500

Agregar al carrito